# yeşilkod

## Green Coding and Digital Carbon Footprint Optimisation Tool

Team: Yunus Emre Sevindik · Mehmet Zahid Görgeç

Measures software energy, fixes the waste, proves the gain. (“Yeşil Kod” is Turkish for “Green Code”.)

PROBLEM

# Data-centre electricity use more than doubles in six years.




Source: IEA, Energy and AI (2025). The 2030 value is the base-case projection.

Part of that energy feeds software waste nobody notices. The energy of a line of code is measured and shown nowhere.

ONE LINE

# How much electricity does this line use?

```
def yoklama_kontrol():              # attendance check
    kayitlilar = list(KAYITLAR)
    return [no for no in GELENLER if no in kayitlilar]
```

**41 kg**

**$CO_2e$ a year, from this line alone.**

One million requests a day, for a year, on the Türkiye grid.

SOLUTION

# You cannot optimise what you do not measure. Yeşil Kod works in four steps.

## 1

### Find

Walks the syntax tree and flags 113 energy-wasting patterns, line by line.

## 2

### Fix

Rewrites eleven of them itself, touching only the changed lines.

## 3

### Verify

Runs both versions and rejects the fix unless the outputs are identical.

## 4

### Measure

Reads CPU energy counters and converts the gain to carbon.

THE FIX

# The engine rewrote the line itself.

```
  def yoklama_kontrol():
      kayitlilar = list(KAYITLAR)
-     return [no for no in GELENLER if no in kayitlilar]
+     kayitlilar_kume = set(kayitlilar)
+     return [no for no in GELENLER if no in kayitlilar_kume]
```

The 2,000-row output is identical in both versions.

**465×**

**less energy, same result.**

Same number of requests, two versions, same computer.

RESULTS

# Four real-world wastes. The engine fixed all four.

| | |
|---|---|
| **Route count**<br>uncached double recursion → lru_cache | ~4,000× |
| **Attendance check**<br>list lookup inside a loop → set() | 465× |
| **Price list**<br>loop-invariant work → moved out of the loop | 51× |
| **Order report**<br>N+1 queries → one batched IN query | 4,001 → 2 queries |

Reduction in CPU energy per request · estimated mode · 26 October 2026 · none of the fixes were written by hand

METHOD

# We measure the gain without overstating it.

## Carbon accounting

$$SCI = (E \times I + M) / R$$

**E** measured CPU energy (powermetrics on Mac, RAPL on Linux)

**I** Türkiye grid, 341 $gCO_2e/kWh$

**M** embodied hardware emissions (next phase)

**R** per request

## Honest limits

- Only CPU energy is measured; memory, network and cooling are excluded. The gain shown is a lower bound.
- Every result is the mean of 5 repeats; caches are cleared before every request.
- In estimated mode, ratios are more reliable than absolute values.

Software Carbon Intensity: Green Software Foundation, ISO/IEC 21031:2024 · Grid: Low-Carbon Power, Aug 2025–Jul 2026

NEXT STEPS

# From prototype to everyday use.

- **More languages:** JavaScript/TypeScript and Java.
- **Full SCI:** embodied emissions and memory energy.
- **Carbon-aware scheduling:** shift heavy jobs to cleaner hours with EPİAŞ hourly data.
- **Developer tools:** editor extension, ready-made CI step.
- **Pilot:** scan student software projects at the university.

**7**
Affordable and clean energy

**9**
Industry, innovation and infrastructure

**12**
Responsible consumption and production

**13**
Climate action

SCAN ME

# Try your own code on your phone.

Paste Python or pick an example, then measure both versions on your own phone. Your code never leaves your device.

→ yes3760.github.io/yesil-kod



yeşilkod

Sakarya University · Yunus Emre Sevindik · Mehmet Zahid Görgeç · Open source: github.com/yes3760/yesil-kod